

NAME: B Sandhya

REG NO: 192311292

CLOUD COMPUTING CASE STUDY ASSESSMENT

CASE STUDY 1: SMART RETAIL INVENTORY MANAGEMENT SYSTEM

1. Understanding of the Case

A retail chain operates multiple stores that need to maintain accurate information about products, stock levels, sales, and customer transactions. Each store uses Point-of-Sale (POS) systems to record purchases and update inventory. Store managers and administrators use dashboards to monitor inventory and business performance.

A cloud-based inventory management system provides a centralized platform where information from all stores can be collected, processed, stored, and analyzed. Instead of maintaining independent databases at every store, the cloud platform provides a common environment for managing product information, transaction records, inventory levels, and analytical data.

The system consists of several important components. Store POS systems act as clients that generate transaction information. Dashboards provide a user interface for managers. APIs provide communication between the clients and cloud services. Cloud databases store structured information, while cloud object storage can store large datasets and logs. Analytics and machine-learning services can analyze historical sales information and forecast future demand.

The system must also provide secure connectivity between stores and the cloud. VPNs, firewalls, API gateways, authentication, authorization, encryption, and monitoring are therefore important components.

The main objective is to create an inventory platform that is centralized, secure, scalable, reliable, and capable of providing real-time business information.

2. Technical Requirements

The system requires several technical components to operate efficiently.

Real-Time Data Collection

POS systems must send sales transactions to the cloud as purchases occur. This allows inventory information to be updated without requiring employees to manually enter transaction information.

API-Based Communication

RESTful or similar web APIs can be used to allow POS systems, dashboards, and other applications to communicate with cloud services.

For example, an inventory API could provide operations such as:

- Retrieve product information.
- Check current stock.
- Update inventory.
- Record a transaction.
- Retrieve sales information.

Cloud Database

A managed cloud database can store structured information such as:

- Product ID
- Product name
- Category
- Price
- Stock quantity
- Store ID
- Customer transaction information
- Supplier information
- Sales records

Cloud Object Storage

Object storage can be used for large or unstructured information such as:

- Transaction logs
- Analytics datasets
- Product images
- Reports
- Backup files

Analytics and Forecasting

The platform requires analytics services to analyze historical sales information. Machine-learning or forecasting services can predict future demand.

Secure Networking

The stores must communicate securely with cloud resources. VPN connectivity, firewalls, secure APIs, and encrypted communication should be used.

Monitoring

The system should monitor application performance, API requests, database activity, system errors, and security events.

3. Business Requirements

The technical system must support the business objectives of the retail organization.

The first requirement is **accurate inventory management**. Store managers should know how many units of a product are currently available.

The second requirement is **reduction of stock-outs**. If a product becomes unavailable, customers may purchase alternatives or leave without purchasing. The system should therefore provide early warnings when inventory reaches a minimum threshold.

The third requirement is **reduction of overstocking**. Excess inventory increases storage costs and can result in products becoming obsolete or unsold.

The fourth requirement is **data-driven purchasing**. Historical sales information and demand forecasts can help managers decide how much stock should be ordered.

The fifth requirement is **centralized management**. Head-office administrators should be able to monitor inventory across multiple stores from a centralized dashboard.

The sixth requirement is **business scalability**. The system should continue to work efficiently when the company opens additional stores or transaction volumes increase.

4. Cloud Computing Concepts Applied

The system can use all three major cloud service models.

Infrastructure as a Service

Virtual machines or cloud compute instances can host applications that require operating-system-level control.

Platform as a Service

Managed databases, analytics platforms, machine-learning services, application runtimes, and other managed services can reduce the amount of infrastructure administration required.

Software as a Service

The inventory dashboard can be provided as a browser-based application that managers can access without installing complex software locally.

Elastic Scalability

Cloud resources can automatically increase during high transaction periods and decrease when demand falls.

Centralized Cloud Storage

Cloud storage provides a central location for information from multiple stores.

Cloud Analytics

Cloud analytics services can process large amounts of historical transaction information without requiring every retail store to maintain its own analytics infrastructure.

5. Working of the System

The operation begins when a customer purchases a product at a store.

The POS system records the transaction and sends the relevant information through a secure API connection.

The API gateway receives the request and performs security checks such as authentication, authorization, validation, and request filtering.

The request is then forwarded to the appropriate backend service.

The transaction service records the purchase, while the inventory service decreases the available quantity of the purchased product.

For example, if a store has 50 units of a product and a customer purchases 3 units, the inventory service updates the available quantity to 47.

The transaction information is stored in the cloud database. Historical transaction data can also be transferred to an analytics environment.

The analytics platform analyzes sales patterns across products, stores, seasons, and time periods.

If the system identifies that a particular product is likely to have increased demand, it can generate a forecast and notify the inventory manager.

The manager can then use the dashboard to make purchasing and replenishment decisions.

6. Security Measures

Security is essential because the system handles business information and potentially sensitive customer and transaction information.

VPN

VPN connections can provide secure communication between retail stores and cloud infrastructure.

Firewall

Firewalls can restrict unauthorized network traffic and allow only approved connections.

API Gateway

The API gateway acts as a controlled entry point for applications communicating with backend services.

It can provide authentication, rate limiting, request validation, monitoring, and routing.

HTTPS/TLS

HTTPS protects information while it travels between POS systems, dashboards, and cloud services.

Authentication

Users and applications must prove their identity before accessing protected resources.

Role-Based Access Control

Different employees should receive different permissions.

For example:

- Store employee: View and update store inventory.
- Store manager: View reports and manage inventory.
- Head-office administrator: Monitor multiple stores.
- System administrator: Manage infrastructure.

Encryption

Sensitive information should be encrypted both during transmission and while stored.

Logging and Monitoring

Security and operational logs should be collected to identify unusual activities, failures, and unauthorized access attempts.

Backup and Recovery

Regular backups should protect transaction and inventory information against accidental deletion or infrastructure failures.

7. Scalability and Availability

The retail platform must support potentially thousands of transactions.

Load balancing can distribute incoming requests among multiple application servers.

Auto-scaling can increase the number of application instances when transaction volume increases.

For example, during a major festival sale, transaction traffic may increase significantly. Additional cloud resources can automatically be allocated to handle the increased demand.

Database replication can improve availability by maintaining multiple copies of important information.

Cloud storage can provide durable storage for logs and datasets.

Multiple availability zones can be used so that failure of one infrastructure location does not necessarily stop the entire application.

8. Analytics and Demand Forecasting

Demand forecasting is one of the most important advantages of the proposed system.

Historical sales data can be analyzed to identify:

- Frequently purchased products.
- Seasonal demand.
- Store-specific purchasing patterns.
- Weekend demand.
- Promotional effects.
- Products with decreasing demand.

For example, if historical data shows that a particular product experiences increased demand every December, the forecasting system can identify this pattern.

Managers can then increase inventory before the demand occurs.

This changes inventory management from a reactive process into a predictive process.

Instead of waiting until a product becomes unavailable, the organization can take action before the shortage occurs.

9. Trade-Off Analysis

A cloud solution provides excellent scalability and flexibility, but it also introduces some disadvantages.

Cloud vs On-Premises

Cloud infrastructure reduces the requirement for the company to purchase and maintain large amounts of physical hardware. However, the organization must continuously pay for cloud resources and depends on network connectivity.

On-premises infrastructure provides greater direct control over hardware, but requires significant investment in servers, storage, cooling, networking, maintenance, and upgrades.

Managed vs Self-Managed Database

A managed database reduces administration and maintenance work. However, it may provide less control over certain infrastructure-level configurations.

A self-managed database provides greater control but requires more technical expertise and maintenance.

Centralized vs Distributed Architecture

Centralized cloud storage simplifies data management, while distributed services can provide better availability and scalability. However, distributed systems are more complex to design and operate.

10. Proposed Solution

The recommended solution is a secure, cloud-based, multi-layer inventory management platform.

Store POS systems and management dashboards communicate through secure APIs. An API gateway provides controlled access to backend services.

Separate services manage inventory, transactions, users, and analytics.

A managed cloud database stores structured business information, while object storage stores logs and large datasets.

Analytics and machine-learning services process historical transactions and generate demand forecasts.

Security is implemented through VPNs, firewalls, HTTPS, authentication, RBAC, encryption, logging, and monitoring.

Load balancing, auto-scaling, database replication, and multiple availability zones provide scalability and availability.

This architecture is practical because it allows the retail organization to start with a manageable infrastructure and expand as the number of stores and transactions increases.

11. Expected Benefits

The system provides several business and technical benefits:

1. Real-time inventory visibility.
2. Improved stock accuracy.
3. Reduced stock-outs.
4. Reduced excess inventory.
5. Better purchasing decisions.
6. Predictive demand forecasting.
7. Centralized management.
8. Improved operational efficiency.
9. Scalable infrastructure.
10. Better security and monitoring.
11. Reduced manual inventory work.
12. Improved customer satisfaction.

CASE STUDY 2: CLOUD-BASED DOCUMENT COLLABORATION SYSTEM

1. Understanding of the Case

The enterprise requires a cloud-based document collaboration platform similar to Google Docs. Users should be able to access documents through browsers and collaborate in real time.

Traditional document sharing often creates multiple copies of the same file. For example, one employee may create a document and send it to another employee through email. The second employee edits the document and sends another copy back. This creates version confusion.

A cloud collaboration platform solves this problem by maintaining a centralized document environment.

Users can open the same document, edit its content, share it with other users, and access it from different devices.

The system requires distributed cloud storage, real-time synchronization, version control, Web APIs, authentication, authorization, encryption, and globally available network infrastructure.

2. Technical Requirements

The platform requires:

- Browser-based client applications.
- Mobile and desktop compatibility.
- Web APIs.
- Real-time communication.
- Distributed document storage.
- Version management.
- User authentication.
- Document permissions.
- Encryption.
- Low-latency communication.
- High availability.
- Backup and disaster recovery.
- Monitoring and logging.

A persistent communication mechanism such as WebSocket can be used for real-time updates.

3. Business Requirements

The main business requirement is to allow employees to collaborate efficiently.

Users should be able to:

- Create documents.
- Open documents.
- Edit documents.
- Share documents.
- Comment on documents.
- View previous versions.
- Restore earlier versions.
- Control document permissions.
- Access documents from different devices.

The platform should also support large numbers of users without significant performance degradation.

For a global enterprise, users located in different countries should experience acceptable response times.

4. Cloud Computing Concepts Applied

Cloud Storage

Documents can be stored in distributed cloud storage systems.

Platform Services

Managed databases, authentication services, messaging systems, and application platforms can reduce infrastructure management.

Content Delivery Network

A CDN can improve performance by delivering frequently accessed content from locations closer to users.

Elastic Computing

Application resources can scale based on the number of active users.

Distributed Architecture

Multiple geographic regions can be used to improve availability and reduce latency.

Software as a Service

The entire collaboration platform can be provided as a cloud-based application accessible through browsers.

5. Working of Real-Time Collaboration

Suppose User A and User B open the same document.

User A changes a sentence.

The change is sent to the collaboration server.

The server determines how the change should be applied to the current document state.

The updated document state is then synchronized with User B.

This process occurs continuously while users are editing.

A real-time communication technology such as WebSocket can maintain a persistent connection between the browser and collaboration service.

This reduces the need for users to repeatedly refresh the page.

6. Conflict Resolution

Multiple users may modify the same document simultaneously.

For example, User A and User B may edit the same paragraph at almost the same time.

A collaboration platform needs a mechanism to resolve such conflicts.

Two important approaches are:

Operational Transformation

Operational Transformation can transform editing operations so that simultaneous modifications remain consistent.

Conflict-Free Replicated Data Types

CRDTs allow distributed systems to represent data in ways that support automatic merging of concurrent changes.

The selected technique depends on the implementation requirements and complexity of the platform.

7. Document Storage and Version Control

Every document should have a unique identifier.

The system can maintain multiple versions of the document.

Example:

Document A

- Version 1 – Initial document.
- Version 2 – First modification.
- Version 3 – Additional content.

- Version 4 – Final version.

The latest version is normally presented to users, while previous versions remain available for recovery.

Version control protects users from accidental modifications and allows organizations to track document history.

8. Security Measures

Encryption in Transit

HTTPS/TLS should protect communication between users and cloud services.

Encryption at Rest

Documents should be encrypted while stored.

Authentication

Users must authenticate before accessing private documents.

Authorization

Authentication identifies the user, while authorization determines what that user can access.

Role-Based Access Control

Different users can receive different permissions.

For example:

Role	Permission
Viewer	Read document
Commenter	Read and comment
Editor	Modify document
Owner	Modify, share and manage permissions
Administrator	Manage enterprise resources

Secure Sharing

Sharing permissions should be controlled to prevent unauthorized users from accessing documents.

Audit Logging

The system should record important events such as:

- Login.
- Document creation.

- Document sharing.
- Editing.
- Permission changes.
- Document deletion.

9. Availability and Low Latency

The system should remain available even when individual servers fail.

Load balancers can distribute requests across application instances.

Multiple application servers can run simultaneously.

If one server becomes unavailable, traffic can be redirected to another healthy server.

For a global organization, multiple geographic cloud regions can be used.

A CDN can reduce latency when delivering frequently accessed static content.

Distributed storage and replication can improve durability and availability.

10. Trade-Off Analysis

Centralized Storage

Centralized storage is easier to manage but may create latency for users who are geographically far from the storage location.

Distributed Storage

Distributed storage improves availability and can reduce latency, but synchronization and consistency become more complex.

WebSockets

WebSockets provide efficient real-time communication but require management of many persistent connections.

Traditional Polling

Polling is easier to implement but can introduce delays and generate unnecessary network requests.

Strong Security

Strong encryption and access controls protect enterprise documents but may introduce some processing overhead.

The security overhead is justified because enterprise documents may contain confidential business information.

11. Proposed Solution

The recommended solution is a globally distributed cloud collaboration platform.

Users connect through browsers or mobile clients over HTTPS.

A CDN and load-balancing layer distributes requests efficiently.

An API gateway controls access to backend services.

Authentication services verify users, while document services manage document operations.

A real-time collaboration service maintains synchronization between users.

Distributed cloud storage stores document files, while a metadata database stores information such as document IDs, owners, permissions, and sharing information.

A version-control service maintains document history and allows previous versions to be restored.

Monitoring, logging, encryption, access control, and backups provide security and reliability.

12. Expected Benefits

The system provides:

1. Real-time collaboration.
2. Centralized document management.
3. Global accessibility.
4. Version history.
5. Reduced duplicate files.
6. Improved employee productivity.
7. Secure document sharing.
8. Better document recovery.
9. High availability.
10. Scalable infrastructure.
11. Reduced dependency on local storage.
12. Easier remote collaboration.

CASE STUDY 3: ONLINE FOOD ORDERING PLATFORM

1. Understanding of the Case

The startup wants to develop an online food ordering platform that customers can access using web browsers and mobile applications.

Customers should be able to discover restaurants, browse menus, select food items, place orders, make payments, and track order information.

Restaurants should be able to manage menus and receive orders.

Delivery personnel should be able to access assigned delivery information.

Administrators should be able to monitor and manage the platform.

Because food ordering traffic can increase significantly during lunch, dinner, weekends, holidays, and promotional campaigns, the system requires elastic scalability.

The proposed technology includes RESTful Web APIs, cloud storage, virtual machines, managed Kubernetes, authentication tokens, HTTPS, and RBAC.

2. Technical Requirements

The technical requirements include:

- Web application.
- Mobile application.
- RESTful APIs.
- API gateway.
- Authentication service.
- Restaurant management.
- Menu management.
- Order management.
- Payment integration.
- Notification service.
- Customer database.
- Cloud object storage.
- Kubernetes cluster.
- Load balancing.
- Auto-scaling.
- Monitoring and logging.
- Backup and recovery.

3. Business Requirements

The platform must provide a simple and reliable customer experience.

Customers should be able to:

- Create accounts.

- Search restaurants.
- View menus.
- Select food.
- Add items to a cart.
- Place orders.
- Make payments.
- View order status.
- Receive notifications.

Restaurants should be able to:

- Register on the platform.
- Add and update menu items.
- Change availability.
- Accept or reject orders.
- Update order status.

Delivery personnel should be able to:

- View assigned orders.
- View delivery information.
- Update delivery status.

Administrators should be able to:

- Manage users.
- Manage restaurants.
- Monitor orders.
- Monitor system activity.
- Handle platform-level issues.

4. Cloud Computing Concepts Applied

Infrastructure as a Service

Virtual machines can host workloads that require control over the computing environment.

Containers

Application services can be packaged into containers so they can be deployed consistently.

Kubernetes

Managed Kubernetes can orchestrate containers and automatically manage service deployment, scaling, and recovery.

Platform as a Service

Managed cloud services can be used for databases, monitoring, storage, and other infrastructure functions.

Object Storage

Menu images, invoices, and other files can be stored using cloud object storage.

Auto-Scaling

Application instances can automatically increase when traffic increases.

5. Working of the Platform

The process begins when a customer opens the web or mobile application.

The client sends a request to the cloud platform through HTTPS.

The request reaches the load balancer and API gateway.

The API gateway identifies the required backend service.

For example, a restaurant search request is sent to the restaurant service.

When the customer selects food and places an order, the order service receives the request.

The order service verifies the user, checks restaurant and menu information, creates the order, and stores the order details in the database.

The restaurant receives the order through its application.

After the restaurant accepts the order, the system updates the order status.

The notification service can notify the customer.

The delivery service can then assign and track delivery information.

6. RESTful API Architecture

REST APIs allow the web and mobile applications to communicate with backend services.

Example operations include:

Restaurant APIs

GET /restaurants

Retrieves available restaurants.

Menu APIs

GET /restaurants/{id}/menu

Retrieves the menu of a particular restaurant.

Order API

POST /orders

Creates a new order.

Order Status

GET /orders/{id}

Retrieves current order information.

Update Order

PUT /orders/{id}

Updates permitted order information.

The API architecture allows multiple clients to use the same backend services.

Therefore, the company does not need to build completely separate backend systems for the web and mobile applications.

7. Kubernetes and Containerization

The platform can divide its backend into independent services.

For example:

- User Service.
- Restaurant Service.
- Menu Service.
- Order Service.
- Payment Service.
- Notification Service.
- Delivery Service.
- Invoice Service.

Each service can run inside containers.

Kubernetes manages these containers.

If an application container fails, Kubernetes can restart it.

If traffic increases, Kubernetes can run additional instances of the service.

For example:

Normal traffic:

Order Service → 2 instances

High traffic:

Order Service → 10 instances

This allows the platform to respond to changing demand.

8. Database and Storage

Structured information should be stored in an appropriate database.

Examples include:

- Customer accounts.
- Restaurant information.
- Menu information.
- Orders.
- Order status.
- Delivery information.

Object storage can be used for:

- Menu images.
- Restaurant images.
- Invoices.
- Promotional images.
- Other large files.

Separating structured database information from object storage makes the system easier to scale.

9. Authentication and Authorization

Authentication confirms who the user is.

Token-based authentication can be used after successful login.

The client sends an access token when making protected API requests.

The backend verifies the token before allowing access.

RBAC determines what each user can perform.

For example:

Customer

Can browse restaurants and place orders.

Restaurant Staff

Can manage menus and restaurant orders.

Delivery Staff

Can access assigned delivery information.

Administrator

Can manage platform-wide resources.

This prevents customers from accessing restaurant administration functions and prevents unauthorized users from modifying platform information.

10. Security Measures

Security is extremely important because the platform handles customer accounts, orders, and payment-related operations.

HTTPS

Protects communication between clients and cloud services.

Authentication Tokens

Protect API endpoints from unauthorized access.

Password Hashing

Passwords should never be stored as plain text.

RBAC

Restricts functionality according to user roles.

API Gateway

Provides a controlled entry point for backend services.

Web Application Firewall

Can help protect web-facing services against common malicious requests.

Input Validation

Prevents invalid or malicious data from reaching application logic.

Rate Limiting

Limits excessive API requests and helps protect services from abuse.

Encryption

Sensitive information should be encrypted appropriately.

Monitoring

System activity, errors, API requests, and security events should be monitored.

11. Scalability

Food ordering platforms can experience sudden traffic increases.

For example, thousands of users may access the application simultaneously during lunch time.

A traditional fixed server architecture may become overloaded.

Cloud infrastructure can solve this using:

- Load balancing.
- Auto-scaling.
- Kubernetes replicas.
- Database scaling.
- Caching.
- CDN.
- Queue-based processing where appropriate.

If the number of users increases, additional application instances can be created.

When traffic decreases, unnecessary instances can be removed.

This provides both performance and cost efficiency.

12. Availability and Reliability

The platform should not depend on a single application server.

Multiple instances should run simultaneously.

If one instance fails, another instance can continue serving requests.

Kubernetes can detect failed containers and restart them.

Load balancers can direct traffic only toward healthy application instances.

Database replication and regular backups can reduce the risk of data loss.

Monitoring systems can alert administrators when services experience problems.

This creates a more reliable platform.

13. Trade-Off Analysis

Virtual Machines vs Kubernetes

Virtual machines provide control and are relatively straightforward, but managing many application instances can become difficult.

Kubernetes provides automated orchestration and scaling but introduces additional complexity.

Monolithic vs Microservices

A monolithic application is simpler to develop initially.

However, all components may need to be scaled together.

Microservices allow individual services to scale independently.

For example, during a major promotion, the Order Service may require more capacity than the User Service.

However, microservices require more monitoring, deployment management, networking, and operational expertise.

Managed vs Self-Managed Kubernetes

Managed Kubernetes reduces infrastructure-management responsibilities but creates greater dependence on the cloud provider.

Self-managed Kubernetes provides greater control but requires significantly more operational work.

For a startup, managed Kubernetes is generally more practical.

14. Proposed Solution

The recommended solution is a cloud-native food ordering platform using web and mobile clients, RESTful APIs, containerized backend services, and managed Kubernetes.

Customers, restaurants, delivery staff, and administrators connect through secure HTTPS connections.

A CDN and load balancer handle incoming traffic.

An API gateway controls and routes API requests.

Independent backend services handle users, restaurants, menus, orders, payments, notifications, and deliveries.

Kubernetes manages the application containers and provides automatic scaling and recovery.

A managed database stores structured business information, while cloud object storage stores menu images and invoices.

Authentication tokens and RBAC protect the APIs.

Monitoring and logging provide visibility into application performance and security events.

This solution is appropriate for a startup because it can begin with a smaller number of services and scale as the number of customers, restaurants, and orders grows.

OVERALL COMPARISON AND ANALYSIS

The three case studies demonstrate different applications of cloud computing.

The **Smart Retail Inventory Management System** focuses on centralized data management, real-time inventory tracking, analytics, and demand forecasting.

The **Cloud-Based Document Collaboration System** focuses on real-time synchronization, distributed storage, version control, global availability, and collaborative editing.

The **Online Food Ordering Platform** focuses on scalable application services, REST APIs, containerization, Kubernetes, secure authentication, and high-volume transaction processing.

CONCLUSION

Cloud computing provides a flexible foundation for all three systems by combining computing resources, storage, networking, APIs, managed services, security mechanisms, and scalable application architectures.

For the Smart Retail system, cloud computing enables centralized inventory management and advanced analytics. The system can use historical sales data to forecast demand and help retailers make better purchasing decisions.

For the Document Collaboration system, distributed cloud infrastructure enables users in different locations to work on documents simultaneously. Real-time synchronization, version control, distributed storage, and access management ensure that collaboration remains efficient and secure.

For the Food Ordering platform, cloud-native technologies such as containers and managed Kubernetes allow the startup to scale application services according to demand. RESTful APIs provide communication between web and mobile clients and backend services, while authentication and RBAC protect the platform.

Although cloud computing introduces challenges such as recurring costs, network dependency, security responsibilities, operational complexity, and potential vendor dependency, the benefits of elasticity, scalability, high availability, centralized management, and rapid deployment make cloud computing highly suitable for these scenarios.

Therefore, all three proposed solutions provide practical approaches for applying cloud computing to real-world business problems while satisfying requirements for **technical functionality, security, scalability, reliability, performance, and business efficiency**.